

CS535/EE514

Machine Learning

Fall 2023

Agha Ali Raza

ENSEMBLE METHODS

ERROR DECOMPOSITION, BAGGING & BOOSTING

Agha Ali Raza



ENSEMBLE METHODS

Error Decomposition

Reading List

1. Machine Learning for Intelligent Systems, Kilian Weinberger, Cornell, Lecture 18 (videos 30, 31), <http://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote18.html>
2. StatQuest with Josh Starmer
 1. StatQuest: Random Forests Part 1 - Building, Using and Evaluating, https://www.youtube.com/watch?v=J4Wdy0Wc_xQ
 2. StatQuest: Random Forests Part 2: Missing data and clustering, https://www.youtube.com/watch?v=nyxTdL_4Q-Q

Setup

Given a dataset $\mathbf{D} = \{(\mathbf{x}_1, \mathbf{y}_1), \dots, (\mathbf{x}_n, \mathbf{y}_n)\}$,

- Drawn i.i.d. from some distribution $\mathbf{P}(\mathbf{X}, \mathbf{Y})$

Assume a regression setting, $\mathbf{y} \in \mathbb{R}$

Goal: Decompose the generalization error of a classifier into interpretable terms.

Strategy: We want to find out the expected squared loss across classifiers $\mathbf{h}$, trained on datasets $\mathbf{D}$ drawn from $\mathbf{P}$, composed of instances $(\mathbf{x}, \mathbf{y})$.

Error Decomposition

$$E_{x,y,D}[(h_D(x) - y)^2] = \underbrace{E_{x,D}[(h_D(x) - \bar{h}(x))^2]}_{\text{Variance}} + \underbrace{E_{x,y}[(\bar{y}(x) - y)^2]}_{\text{Noise}} + \underbrace{E_x[(\bar{h}(x) - \bar{y}(x))^2]}_{\text{Bias}^2}$$

Where,

For a particular ML algorithm, **A** (e.g., decision trees):

- $h_D(x)$ is the predicted output for input x of a model h trained on dataset D
 - Remember D was sampled from the distribution P , i.e., $D \sim P$
- y is the actual label for x
- $(h_D(x) - y)^2$ is the squared error between the hypothesized and actual output
- $E_{x,y,D}[(h_D(x) - y)^2]$ is the expected error of the classifiers h_D across all possible combinations of (x, y) and trained on all possible datasets D drawn from P .

Error Decomposition: Variance

$$E_{x,y,D}[(h_D(x) - y)^2] = \underbrace{E_{x,D}[(h_D(x) - \bar{h}(x))^2]}_{\text{Variance}} + \underbrace{E_{x,y}[(\bar{y}(x) - y)^2]}_{\text{Noise}} + \underbrace{E_x[(\bar{h}(x) - \bar{y}(x))^2]}_{\text{Bias}^2}$$

Where,

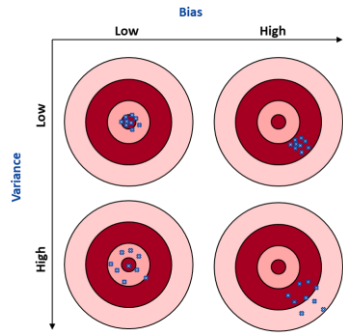
- $h_D(x)$ is the predicted output for input x of a model h trained on dataset D
- $\bar{h}$ is the expected classifier (given ML algorithm, $\mathbf{A}$ (e.g., decision trees)) – the model that is averaged across all models trained on all possible datasets D weighted by the probability of the draw:

$$\bar{h} = E_{D \sim P^n} [h_D] = \int_D h_D \text{Pr}(D) \partial D$$

- where $\text{Pr}(D)$ is the probability of drawing D from P^n
- Here, $\bar{h}$ is a weighted average over functions h_D
- $(h_D(x) - \bar{h}(x))^2$ is the squared difference between the classifier trained on D and the average classifier
- $E_{x,D}[(h_D(x) - \bar{h}(x))^2]$ is the error between the average model and our model as it is trained across all possible datasets D and tested on all possible inputs x .

Variance

- How much does our model $h_D(x)$ change if we train on different training sets, D ? (drawn from the same P)
- How much is our model likely to overfit to each training set D ?
- How far off does our model remain from the average classifier, $\bar{h}(x)$ - on average?



Error Decomposition: Noise

$$E_{x,y,D}[(h_D(x) - y)^2] = \underbrace{E_{x,D}[(h_D(x) - \bar{h}(x))^2]}_{\text{Variance}} + \underbrace{E_{x,y}[(\bar{y}(x) - y)^2]}_{\text{Noise}} + \underbrace{E_x[(\bar{h}(x) - \bar{y}(x))^2]}_{\text{Bias}^2}$$

Where,

- $\bar{y}(x)$ is the average of the true labels for the input x . It is the best possible predicted label.
- y is the actual gold label for x
- $E_{x,y}[(\bar{y}(x) - y)^2]$ is the expected error between the best label that could be predicted and the actual label.

Noise

- How large is the data-intrinsic noise?
 - Noisy data: Incorrect labels or different labels for the same set of inputs
- Measures ambiguity due to data distribution and feature representation.
 - Two houses with different prices for the same set of inputs across the features. However, an important feature “location” is missing.
- Nothing much can be done about it. It is an aspect of the data.

Error Decomposition: Bias

$$E_{x,y,D}[(h_D(x) - y)^2] = \underbrace{E_{x,D}[(h_D(x) - \bar{h}(x))^2]}_{\text{Variance}} + \underbrace{E_{x,y}[(\bar{y}(x) - y)^2]}_{\text{Noise}} + \underbrace{E_x[(\bar{h}(x) - \bar{y}(x))^2]}_{\text{Bias}^2}$$

Where,

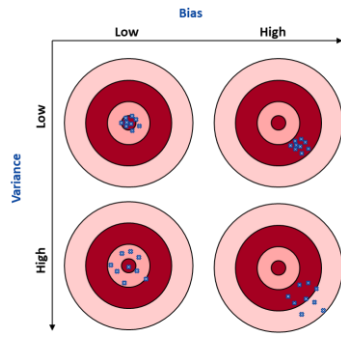
- $\bar{h}$ is the expected classifier (given ML algorithm, $\mathbf{A}$ (e.g., decision trees)) – the model that is averaged across all models trained on all possible datasets D weighted by the probability of the draw:

$$\bar{h} = E_{D \sim P^n} [h_D] = \int_D h_D \Pr(D) dD$$

- where $\Pr(D)$ is the probability of drawing D from P^n
- Here, $\bar{h}$ is a weighted average over functions h_D
- $\bar{y}(x)$ is the average of the true labels for the input x . It is the best possible predicted label.
- $E_x[(\bar{h}(x) - \bar{y}(x))^2]$ is the expected square difference between the best label that our ML algorithm, $\mathbf{A}$ can produce ($\bar{h}(x)$) and the best possible label that could be predicted from the data ($\bar{y}(x)$).

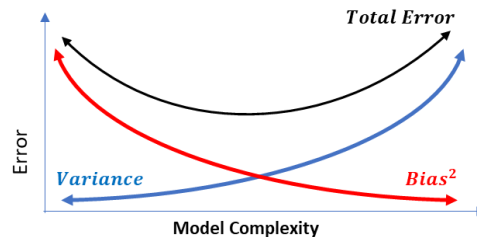
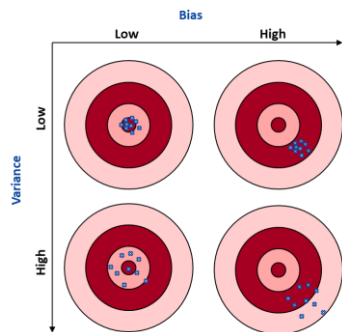
Bias

- What is the inherent error that we obtain from our classifier even with infinite training data?
- This is due to our classifier being "biased" to a particular kind of solution (e.g., linear classifier).
- Bias is inherent to our model.



Techniques to Reduce Variance / Bias

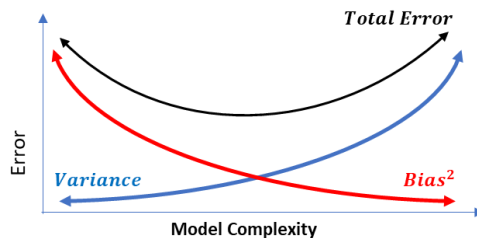
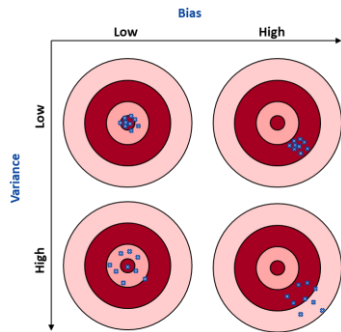
The Bias-Variance Tradeoff



High Variance, Low Bias

- The model has overfitted to the training data.
- Strong Learners
- **Symptoms:**
 - Training error is much lower than test error
 - Training error is within an acceptable range
 - Test error is above the acceptable limit
- **Remedies:**
 - Add more training data – as the model has already overfitted to the existing data
 - Reduce model complexity -- complex models are prone to high variance
 - **Bagging**

The Bias-Variance Tradeoff



High Bias, Low Variance

- The model is not robust enough to produce an accurate prediction.
- Weak learners
- **Symptoms:**
 - The training error is higher than the acceptable limit
- **Remedies:**
 - Use more complex models (e.g., kernelize, use non-linear models)
 - Add features
 - **Boosting**

Bagging Intuition

The Wisdom of Crowds

- Observers are asked to estimate the number of pennies in a glass jar.
 - Individuals do not do well, but collectives do remarkably well.
 - Some overestimate, others underestimate – averaged judgments are quite accurate.
- **Mechanism**
 - All judgments have a common basis.
 - Individual errors are independent, and in the absence of a systematic bias, they average to zero.
 - It only works when the observations are independent and their errors are uncorrelated.
 - If the observers share a bias, the aggregation of judgments will not reduce it.



The Wisdom of Crowds by James Surowiecki

Thinking, Fast and Slow by Daniel Kahneman

The Wisdom of Crowds

- Allowing the observers to influence each other effectively reduces the sample size.
- The principle of independent judgments (and de-correlated errors) has immediate applications for the conduct of meetings.
- A simple rule:
 - Before an issue is **discussed**, all members write a summary of their position.
 - This makes good use of the value of the **diversity** of knowledge and opinion.
 - The standard practice of open discussion gives too much weight to the opinions of those who speak early and assertively, causing others to line up behind them.

The Wisdom of Crowds by James Surowiecki

Thinking, Fast and Slow by Daniel Kahneman

ENSEMBLE METHODS

Variance Reduction – Theory

Setting Our Goals

$$E_{x,y,D}[(h_D(x) - y)^2] = \underbrace{E_{x,D}[(h_D(x) - \bar{h}(x))^2]}_{\text{Variance}} + \underbrace{E_{x,y}[(\bar{y}(x) - y)^2]}_{\text{Noise}} + \underbrace{E_x[(\bar{h}(x) - \bar{y}(x))^2]}_{\text{Bias}^2}$$

- Where $\bar{h}$ is the expected classifier (given ML algorithm A):

$$\bar{h} = E_{D \sim P^n} [h_D] = \int_D h_D \Pr(D) \partial D$$

- where $\Pr(D)$ is the probability of drawing D from P^n
- Here, $\bar{h}$ is a weighted average over functions h_D

- Goal:** Reduce the variance term: $E_{x,D}[(h_D(x) - \bar{h}(x))^2]$

- By making $h_D \rightarrow \bar{h}$.

Using the weak law of large numbers

- For i.i.d. random variables x_i with mean $\bar{x}$, the weak law roughly states:

$$\frac{1}{m} \sum_{i=1}^m x_i \rightarrow \bar{x} \text{ as } m \rightarrow \infty$$

- **In case of classifiers:**

- Assume we have m training sets $D_1, D_2, \dots, D_m$ drawn from P^m
- Train a classifier on each one and average out the results

$$\hat{h} = \frac{1}{m} \sum_{i=1}^m h_{D_i} \rightarrow \bar{h} \text{ as } m \rightarrow \infty$$

- Such an average of multiple classifiers is referred to as an **ensemble of classifiers**
- **Success!** ☺ If $\hat{h} \rightarrow \bar{h}$, the variance component of the error must also vanish

$$E_{x,D} \left[\left(h_D(x) - \bar{h}(x) \right)^2 \right] \rightarrow 0$$

- **Bummer!** ☹ We don't have m data sets. We only have our lonely old D .

ENSEMBLE METHODS

Bagging – Setup

Bagging (**B**ootstrap **A**ggregating)

Simulate drawing from P by drawing **uniformly with replacement** from the set D

- We get $D_1, D_2, \dots, D_n$ — each one with the same size $|D|$
- Use these datasets to train n classifiers, $h_1, h_2, \dots, h_n$
 - completely all the way to a minimum error on the training data i.e., overfitting
- Finally, average out the decisions of these n classifiers (in case of regression) or take the mode (majority vote, in case of classification)



Bagging (Bootstrap Aggregating)

Okay, so now, can we claim that $\hat{h} \rightarrow \bar{h}$ using the weak law of large numbers?

- Nope!
- Bagged classifier:

$$\hat{h} = \frac{1}{m} \sum_{i=1}^m h_{D_i} \nrightarrow \bar{h}$$

- As, we cannot use W.L.L.N here
- W.L.L.N only works for i.i.d. samples and here we violate the independence assumption – knowing a few of D'_i 's tells us about D_{i+1} as all of these come from the same dataset D containing the same finite set of points
- Can we even show that these points have been drawn from the same distribution P ?
 - Yes! I have omitted the proof, though.

However, **in practice** bagging still reduces variance **very effectively**.

- If each of $h_1, h_2, \dots, h_n$ overfitted to some unique aspects of $D_1, D_2, \dots, D_n$, the average classifier will average out these over-reactions and preserve the commonalities across them that supposedly represent D , and in turn, P

Summary of Bagging

1. Sample m data sets $D_1, \dots, D_m$ from D with replacement.
2. For each D_j train a classifier h_j
3. The final classifier is $h(x) = \sum_{j=1}^m h_j(x)$
4. In practice, larger m results in a better ensemble:
 - However, at some point we will obtain diminishing returns
 - Setting m unnecessarily high will only slow down the classifier but will not increase the error of the classifier.
 - Bagging is a general technique for reducing variance and not just specific to decision trees

ENSEMBLE METHODS

Bagging – Advantages

Advantages of Bagging

- Easy to implement.
- Reduces variance, so it is great for high variance classifiers.
- As the prediction is an average (or mode) of many classifiers, we obtain a mean and variance.
 - The variance can be interpreted as the uncertainty of the prediction. Especially in regression tasks
 - Such uncertainties are otherwise hard to obtain.
 - Imagine the prediction of a house price is \$300,000. If a buyer wants to decide how much to offer, it would be very valuable to know the variance of the prediction.

Advantages of Bagging

- Bagging provides an unbiased estimate of the test error – the **out-of-bag error**.
- Take each training point (x_i, y_i) that was not picked by the data sets D_k . If we average the classifiers h_k of all such data sets, we obtain a classifier (with a slightly smaller m) that was not trained on (x_i, y_i)
 - (x_i, y_i) is therefore equivalent to a test sample.
- Compute the average error of the all such ensembles of classifier to obtain an estimate of the **true test error**.
- The beauty is that we can do this without reducing the training set. We just run bagging as it is intended and obtain this so called **out-of-bag error** for free.
- If m is sufficiently large, the fact that we take out some classifiers has no significant effect, and the estimate is reliable.

ENSEMBLE METHODS

Random Forests

Sources

- **Machine Learning for Intelligent Systems**, Kilian Weinberger, Cornell, Lecture 18 (videos 30, 31), <http://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote18.html>
- **StatQuest with Josh Starmer**
 - StatQuest: Random Forests Part 1 - Building, Using and Evaluating, https://www.youtube.com/watch?v=J4Wdy0Wc_xQ
 - StatQuest: Random Forests Part 2: Missing data and clustering, https://www.youtube.com/watch?v=nyxTdL_4Q-Q

Random Forest

- One of the most famous and useful bagged algorithms is the **Random Forest**
 - A Random Forest is essentially just bagged decision trees, with a slightly modified splitting criteria.
- **Algorithm**
 1. Sample m data sets $D_1, \dots, D_m$ from D with replacement
 2. For each D_j train a full decision tree h_j ($maxdepth \rightarrow \infty$) with one small modification:
 - Before each split randomly subsample $k \leq d$ features (without replacement) and only consider these for your split
 - This further increases the variance of the trees – it makes the trees more different. Now each tree makes slightly different mistakes, and these get averaged out later
 3. The final classifier is:

$$h(x) = \frac{1}{m} \sum_{j=1}^m h_j(x)$$

ENSEMBLE METHODS

Random Forests – Advantages

Benefits

The Random Forest is one of the best, most popular and easiest to use **out-of-the-box** classifier

1. RF has **only two hyper-parameters**, m and k
 - It is extremely **insensitive to both**
 - A good choice for k is $k = \sqrt{d}$ (where d denotes the number of features).
 - Set m as large as you can afford

Benefits

2. All the benefits of trees get passed down to RF
 - Decision trees do not require a lot of preprocessing. **No need to normalize/scale features for mixture of feature types and ranges**
 - For example, the features can be of different scale, magnitude, or slope. This can be highly advantageous in scenarios with heterogeneous data, for example the medical settings where features could be things like blood pressure, age, gender, ..., each of which is recorded in completely different units.
 - **Feature selection:** When we split on a feature, how much does the entropy go down. Take average across trees in the forest and sort.
 - **Resilient to the curse-of-dimensionality**
3. **Out-of-bag error** as a great estimator of test error – no need for training/validation splits. Train on the entire dataset!

ENSEMBLE METHODS BIAS REDUCTION – THEORY

Sources

- **Machine Learning for Intelligent Systems**, Kilian Weinberger, Cornell, Lecture 18 (videos 32, 33, 34), <https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote19.html>
- **StatQuest with Josh Starmer**
 - Statquest: Gradient Boost Part 1 (of 4): Regression Main Ideas, <https://www.youtube.com/watch?v=3CC4N4z3GJc>
 - Statquest: Gradient Boost Part 2 (of 4): Regression Details, <https://www.youtube.com/watch?v=2xudPOBz-vs>
 - Statquest: Gradient Boost Part 3 (of 4): Classification, <https://www.youtube.com/watch?v=jxuNLH5dXCs>
 - Statquest: Gradient Boost Part 4 (of 4): Classification Details, <https://www.youtube.com/watch?v=StWY5QWMXCw>
 - StatQuest: AdaBoost, Clearly Explained, <https://www.youtube.com/watch?v=LsK-xG1cLYA>

The Bias Problem

$$E_{x,y,D}[(h_D(x) - y)^2] = \underbrace{E_{x,D}[(h_D(x) - \bar{h}(x))^2]}_{\text{Variance}} + \underbrace{E_{x,y}[(\bar{y}(x) - y)^2]}_{\text{Noise}} + \underbrace{E_x[(\bar{h}(x) - \bar{y}(x))^2]}_{\text{Bias}^2}$$

- Where the Bias term is the square of the difference between the prediction of our classifier ($\bar{h}(x)$) and the best prediction that could be learned ($\bar{y}(x)$)
- **Scenario:** Classifiers have a large bias, and the training error is high (e.g., CART trees with very limited depth.)

Goal: Reduce the bias term: $E_x[(\bar{h}(x) - \bar{y}(x))^2]$

- By making $\bar{h}(x) \rightarrow y$

$$\bar{h}(x) = H_T(\vec{x}) = \sum_{t=1}^T \alpha_t h_t(\vec{x})$$

- Where $H_T(\vec{x})$ is the ensemble classifier

Boosting

- **Basic idea:** Combine weak learners to generate a strong learner with low bias.
- Iteratively, create an ensemble classifier.

$$\bar{h}(x) = H_T(\vec{x}) = \sum_{t=1}^T \alpha_t h_t(\vec{x})$$

- In iteration t add the classifier $\alpha_t h_t(\vec{x})$ to the ensemble.
 - Choose the h_t that results in the best reduction of the **remaining error – the residual error**
 - **For non-adaptive boosting**, use the same value of α_t for all t
 - **For adaptive boosting (Ada Boost)**, give more weight to the h_t that lead to higher reduction in the remaining error
- At test time, evaluate all classifiers and return the weighted sum.
 - Use it as it is for regression
 - Compare the sum across categories and let the category with the highest weighted sum win, in case of classification
- The process of constructing such an ensemble in a stage-wise fashion is very similar to **gradient descent**.
 - Instead of updating the model parameters in each iteration, we add functions to our ensemble
 - **Boosting is actually gradient descent in function space**

Intuition

- We have a weak learner $h(x) \in H$
 - It cannot reduce the training error significantly
 - E.g., Just a bit better than a coin toss
- How can we use $h(x)$ to reduce the training error?
- For simplicity, assume a regression setting.
 - $D = \{(x_1, 7), (x_2, 9)\}$,
 - where $X = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$ are the features and $Y = \begin{bmatrix} 7 \\ 9 \end{bmatrix}$ are the labels
 - $h(X)$ returns $\begin{bmatrix} h(x_1) \\ h(x_2) \end{bmatrix}$
 - The Half of the Squared Error would be defined as:

$$l(h) = \frac{1}{2} \sum_{i=1}^n (h(x_i) - y_i)^2$$

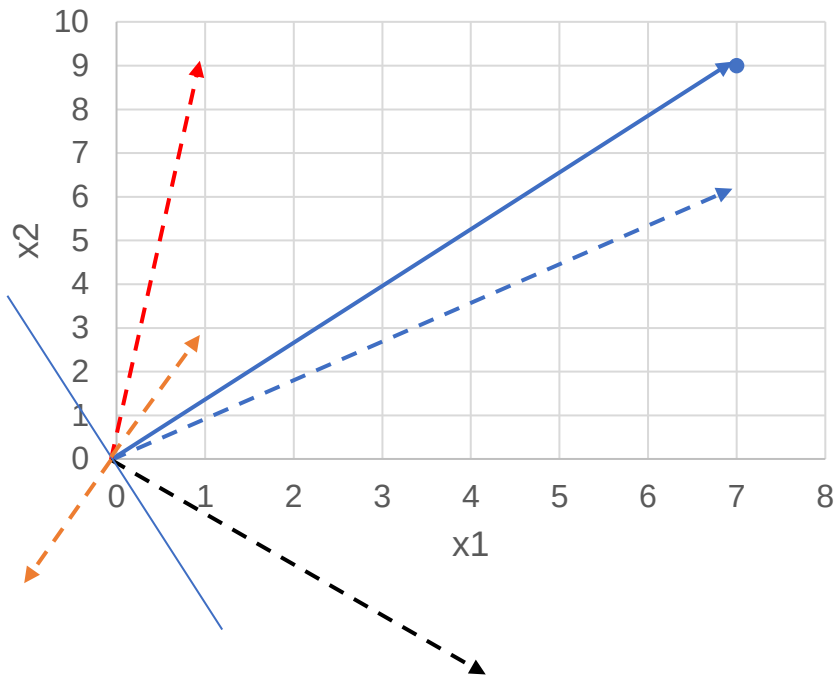
- Suppose, $h_1(X) = \begin{bmatrix} 6 \\ 4 \end{bmatrix}$, the squared error is:

$$\begin{aligned} l(h_1) &= \frac{1}{2} ((6 - 7)^2 + (4 - 9)^2) \\ &= \frac{1}{2} (1 + 25) = 13 \end{aligned}$$

- Let's set out in search of the weak learners to be added to our ensemble!

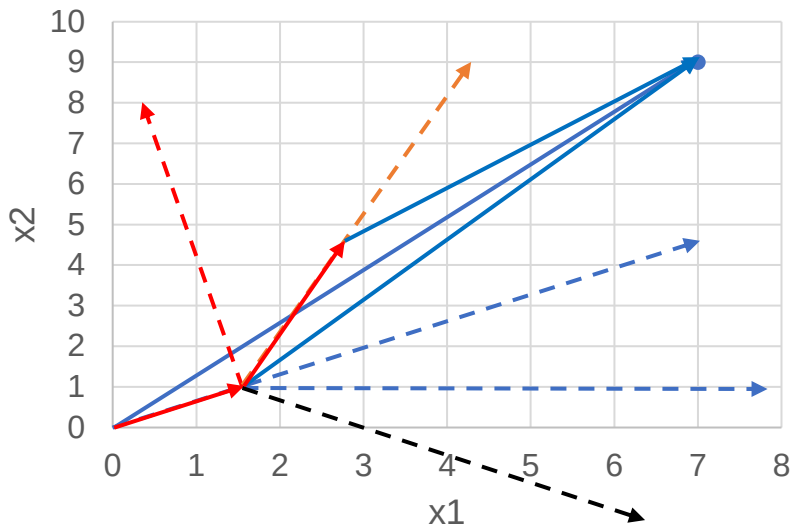
Intuition

- **Scenario:** Asking silly guides for directions
- **Goal:** Find the best weighted line-up of silly guides to a destination



Intuition

- **Scenario:** Asking silly guides for directions
- **Goal:** Find the best weighted line-up of silly guides to a destination



$$H_T(\vec{x}) = \sum_{t=1}^T \alpha_t h_t(\vec{x})$$

The gradient of the loss function w.r.t $H(x_i)$

$$l(H) = \frac{1}{2} \sum_{i=1}^n (H(x_i) - y_i)^2$$
$$\frac{\partial l}{\partial H(x_i)} = H(x_i) - y_i$$

Which is a vector from the destination to our current position.
Flipping it, we get our compass:

$$\text{Let } t_i = -\frac{\partial l}{\partial H(x_i)} = y_i - H(x_i)$$

Which are the residuals along each training instance!

- Also, the vector from H to Y
- So, we are looking for an h with the minimum loss with t .

ENSEMBLE METHODS

Gradient-boosted Regression Trees

Sources

- **Machine Learning for Intelligent Systems**, Kilian Weinberger, Cornell, Lecture 18 (videos 32, 33, 34), <https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote19.html>
- **StatQuest with Josh Starmer**
 - Statquest: Gradient Boost Part 1 (of 4): Regression Main Ideas, <https://www.youtube.com/watch?v=3CC4N4z3GJc>
 - Statquest: Gradient Boost Part 2 (of 4): Regression Details, <https://www.youtube.com/watch?v=2xudPOBz-vs>
 - Statquest: Gradient Boost Part 3 (of 4): Classification, <https://www.youtube.com/watch?v=jxuNLH5dXCs>
 - Statquest: Gradient Boost Part 4 (of 4): Classification Details, <https://www.youtube.com/watch?v=StWY5QWMXCw>
 - StatQuest: AdaBoost, Clearly Explained, <https://www.youtube.com/watch?v=LsK-xG1cLYA>

Gradient Boosted Regression Trees

- Classification ($y_i \in \{+1, -1\}$) or regression ($y_i \in R^k$)
- Weak learners, $h \in H$, are regressors
$$h(x) \in R, \forall x$$
- typically fixed-depth (e.g., depth=4) regression trees
- Step size α is fixed to a small constant – hyper-parameter
- **Loss function:** Any differentiable convex loss that decomposes over the samples

$$L(H) = \sum_{i=1}^n \ell(H(x_i))$$

Gradient Boosted Regression Trees

Algorithm

Input: $\ell, \alpha, \{(x_i, y_i)\}, A$

$H = 0$

for $t = 1:T$ *do*

$\forall i: t_i = y_i - H(x_i)$

$h = \operatorname{argmin}_{h \in H} \sum_{i=1}^n (h(x_i) - t_i)^2$

$H \leftarrow H + \alpha h$

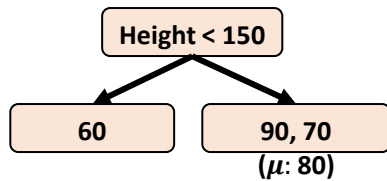
end

return H

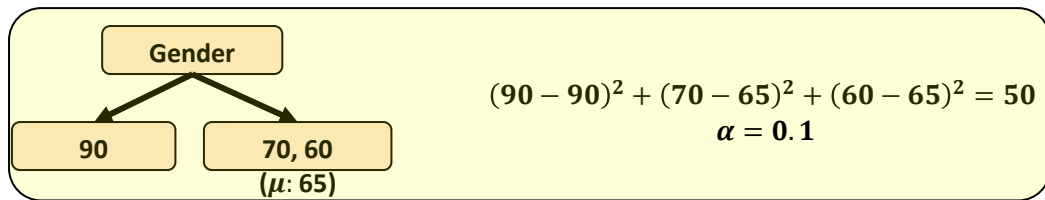
ENSEMBLE METHODS

Gradient-boosted Trees – Example

Height (cm)	Gender	H.B.P	Weight (kg)
170	Male	Yes	90
160	Female	No	70
140	Female	Yes	60

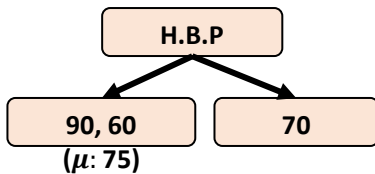


$$(90 - 80)^2 + (70 - 80)^2 + (60 - 60)^2 = 200$$



$$(90 - 90)^2 + (70 - 65)^2 + (60 - 65)^2 = 50$$

$$\alpha = 0.1$$



$$(90 - 75)^2 + (70 - 70)^2 + (60 - 75)^2 = 450$$

Input: $\ell, \alpha, \{(x_i, y_i)\}, A$

$H = 0$

for $t = 1:T$ do

$\forall i: t_i = y_i - H(x_i)$

$h = \operatorname{argmin}_{h \in H} \sum_{i=1}^n (h(x_i) - t_i)^2$

$H \leftarrow H + \alpha h$

end

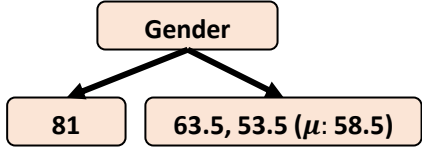
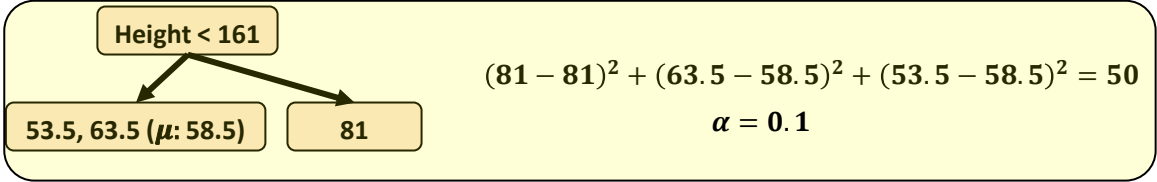
return H

Height (cm)	Gender	H.B.P	Weight (kg)	H(x)	Residuals	H(x)+ah	Residuals
170	Male	Yes	90	0	90	9	81
160	Female	No	70	0	70	6.5	63.5
140	Female	Yes	60	0	60	6.5	53.5

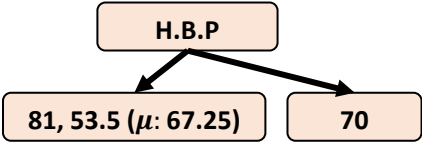
```

Input:  $\ell, \alpha, \{(x_i, y_i)\}, A$ 
 $H = 0$ 
for  $t = 1:T$  do
     $\forall i: t_i = y_i - H(x_i)$ 
     $h = \operatorname{argmin}_{h \in H} \sum_{i=1}^n (h(x_i) - t_i)^2$ 
     $H \leftarrow H + ah$ 
end
return  $H$ 

```



$$(81 - 81)^2 + (63.5 - 58.5)^2 + (53.5 - 58.5)^2 = 50$$



$$(81 - 67.25)^2 + (63.5 - 63.5)^2 + (53.5 - 67.25)^2 = 378.125$$

ENSEMBLE METHODS

ADABOOST

Sources

- **Machine Learning for Intelligent Systems**, Kilian Weinberger, Cornell, Lecture 18 (videos 32, 33, 34), <https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote19.html>
- **StatQuest with Josh Starmer**
 - Statquest: Gradient Boost Part 1 (of 4): Regression Main Ideas, <https://www.youtube.com/watch?v=3CC4N4z3GJc>
 - Statquest: Gradient Boost Part 2 (of 4): Regression Details, <https://www.youtube.com/watch?v=2xudPOBz-vs>
 - Statquest: Gradient Boost Part 3 (of 4): Classification, <https://www.youtube.com/watch?v=jxuNLH5dXCs>
 - Statquest: Gradient Boost Part 4 (of 4): Classification Details, <https://www.youtube.com/watch?v=StWY5QWMXCw>
 - StatQuest: AdaBoost, Clearly Explained, <https://www.youtube.com/watch?v=LsK-xG1cLYA>

Adaptive Boosting (Ada Boost)

- Each tree is only grown to the minimal height
 - i.e., Ada Boost combines weak learners (stumps)
- Each tree gets a proportional say (α) based on its performance
- Each instance gets assigned a weight (equal in the beginning)
 - The weights of misclassified instances get amplified while those of correctly classified ones get reduced.
- The order of the construction of stumps is important as each stump tries to reduce the **residual weighted error**

ENSEMBLE METHODS

Summary of Boosting

Boosting

- Boosting turns a crowd of weak classifiers into a strong classifier
 - Each barely better than a coin-toss
- Each weak learner just need to be a bit better than chance – a coin toss
- Gradient Boosted Regression Trees is one of the most popular algorithms for page ranking for search engines
- Boosting is resilient to variance
 - Bias gets reduced quickly but the variance increases slowly